

Lista de exercícios — SAP-1 (com gabarito)

Responda **antes** de olhar o gabarito (no fim). Estudar respondendo é o que mais fixa. Se travar numa questão, o gabarito diz onde revisar.

Parte A — Componentes

- A1.** O que o PC (contador de programa) guarda? O que o sinal `Cp` faz?
- A2.** Qual a diferença entre o **MAR** e o **IR**? (o que cada um guarda?)
- A3.** A ALU **guarda** o resultado da soma? Por que, na onda, o sinal `result` fica "mudando" mesmo sem ninguém mandar?
- A4.** Por que o registrador B é chamado de "descartável"?

Parte B — O barramento e os enables

- B1.** Por que só **um** bloco pode "falar" (colocar valor) no barramento por vez?
- B2.** Cite os **5 sinais de "falar"** e o que cada um coloca no barramento.
- B3.** No estado T3, quem fala e quem ouve? Qual é a transferência?

Parte C — Palavra de controle

- C1.** Por que o estado de repouso (IDLE) é `0011_1110_0011` e **não** `0000_0000_0000` ?
- C2.** Decodifique `0010_1110_0001` : quais sinais estão ativos e qual a ação? (dica: compare com o IDLE)
- C3.** Decodifique `0011_1100_1111` : quais sinais e qual ação?

Parte D — Estados e ciclo

- D1.** Quantos estados uma instrução gasta? Quais são de **busca** e quais de **execução**?
- D2.** Em qual estado o PC incrementa? Em qual o IR é carregado?
- D3.** No `LDA` , em qual estado o acumulador A recebe o valor? E no `ADD` ? (por que é diferente?)
- D4.** Por que o processador congela em **T4** quando encontra o `HLT` ?

Parte E — Montar e traçar programas

- E1.** LDA 11 carrega o valor 3. Por quê? O que você mudaria para ele carregar 7?
- E2.** Escreva o binário (opcode|operando) destas instruções: LDA 13 , SUB 14 , OUT , HLT .
- E3.** Monte um programa que calcula $6 - 2 = 4$ e mostra o resultado. Dê o conteúdo das posições de memória usadas (em binário) e trace o acumulador.

Parte F — Ler a onda

- ✓**F1.** Na janela Wave, qual sinal você segue para "ver a conta acontecer"?
- F2.** Por que o out_port (resultado) fica **parado** entre dois OUT ?

Gabarito

- A1.** O PC guarda o **endereço da próxima instrução**. Cp faz o PC **incrementar** (+1) na próxima borda de subida. (rev.: componentes.md §1)
- A2.** O **MAR** guarda o **endereço** que a RAM vai ler (4 bits). O **IR** guarda a **instrução** lida (8 bits) e a divide em opcode + operando.
- A3.** Não. A ALU é **combinacional**: ela sempre mostra $A \pm B$ do momento. Como A e B mudam ao longo da execução, result acompanha — mas só vale quando Eu joga no barramento (no T6). (rev.: componentes.md §6)
- A4.** Porque ele é recarregado a cada ADD / SUB (no T5) e **não guarda resultado** — é só o "segundo número" da conta do momento.
- B1.** Porque é **um fio só**. Se dois falassem juntos, os valores colidiriam (curto/indefinido). Por isso o controlador liga só **um falante** por estado. (rev.: fluxo_completo.md, Parte 1)
- B2.** $E_p \rightarrow PC \cdot \sim CE \rightarrow RAM \cdot \sim Ei \rightarrow \text{operando do IR} \cdot Ea \rightarrow \text{acumulador} \cdot Eu \rightarrow \text{saída da ALU}$.
- B3.** No T3: a **RAM** fala ($\sim CE$) e o **IR** ouve ($\sim Li$). Transferência: **IR ← RAM** (busca da instrução).
- C1.** Porque há sinais **ativos-baixo** (com til): eles ficam em **1** quando desligados. Então o repouso tem os ativos-alto em 0 e os ativos-baixo em 1 → 0011_1110_0011 . (rev.: palavra_controle.md §2)
- C2.** $\sim CE = 0$ e $\sim Lb = 0 \rightarrow$ **RAM fala, B ouve** → **B ← RAM**. É o **T5 do ADD/SUB**.
- C3.** $\sim La = 0$, $Eu = 1$, $Su = 1 \rightarrow$ **A ouve, ALU fala, ALU subtrai** → **A ← A – B**. É o **T6 do SUB**.
- D1. 6 estados** (T1–T6). **Busca:** T1, T2, T3 (iguais p/ toda instrução). **Execução:** T4, T5, T6 (dependem do opcode).

D2. PC incrementa no **T2**. IR é carregado no **T3**.

D3. No **LDA**, A recebe no **T5**. No **ADD / SUB**, A recebe no **T6** (porque primeiro precisa carregar B no T5 e só então somar). Por isso, na onda, o A do ADD só muda na virada **T6 → T1 seguinte**.

D4. O **HLT** é detectado no T4 (o opcode já está no IR desde o T3). Nesse ponto trava **halted=1**, o contador de anel para de avançar e fica preso em **T4**. Não há o que fazer depois disso.

E1. Porque **11** é um **endereço**, não um valor — e na posição 11 está guardado o dado **3** (**mem[11]=3**). Para carregar 7, basta pôr **mem[11]=7** (ou apontar para um endereço que contenha 7). (*endereçamento direto*)

E2.

```
LDA 13 = 0000 1101
SUB 14 = 0010 1110
OUT   = 1110 0000
HLT   = 1111 0000
```

E3. Uma solução (dados em 14 e 15):

```
mem[0] = 0000_1110 ; LDA 14  -> A = 6
mem[1] = 0010_1111 ; SUB 15  -> A = 6-2 = 4
mem[2] = 1110_0000 ; OUT      -> mostra 4
mem[3] = 1111_0000 ; HLT
mem[14] = 0000_0110 ; dado = 6
mem[15] = 0000_0010 ; dado = 2
```

Traço do A: **0 → 6 → 4**. Resultado: **4** (**0x04**).

F1. A linha do **acumulador acc_out** (verde no **wave_sap1.do**). É onde a conta aparece subindo/descendo.

F2. Porque o **out_port** é o **registrador de saída**, que só carrega quando roda um **OUT** (**~Lo** ativo). Entre dois **OUT**, ele mantém o valor — por isso fica parado.

Desafios extras (sem gabarito — teste-se!)

1. Monte **2 × 5** por somas repetidas (resultado 10) e simule.
2. Decodifique **0101_1110_0011** e diga o estado. (*dica: é o T1*)
3. Sem simular, preveja a saída de: **LDA 15, ADD 15, ADD 15, OUT, HLT** com **mem[15]=4**. (*resposta: 12*)